

Rockchip RK3399 Linux SDK Quick Start

ID: RK-FB-YF-944

Release Version: V1.5.0

Release Date: 2024-06-20

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

The document presents Rockchip RK3399 Linux SDK release notes, aiming to help engineers get started with RK3399 Linux SDK development and debugging faster.

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Chipset and System Support

Chipset	Buildroot	Debian	Yocto
RK3399	Y	Y	Y

Revision History

Date	Version	Author	Revision History
2022-06-20	V1.0.0	Caesar Wang	Initial version
2022-09-20	V1.0.1	Caesar Wang	Update it for linux5.10
2022-11-20	V1.0.2	Caesar Wang	Update Linux Upgrade Instruction
2023-04-20	V1.1.0	Caesar Wang	Adapt to the new version of SDK
2023-05-20	V1.1.1	Caesar Wang	Update SDK to V1.1.1
2023-06-20	V1.2.0	Caesar Wang	Update SDK to V1.2.0
2023-09-20	V1.3.0	Caesar Wang	Update SDK to V1.3.0
2023-12-20	V1.4.0	Caesar Wang	Update SDK to V1.4.0
2024-06-20	V1.5.0	Caesar Wang	Update SDK to V1.5.0

Contents

Rockchip RK3399 Linux SDK Quick Start

1. SDK Precompiled Image
2. Set up an Development Environment
 - 2.1 Preparing the development environment
 - 2.2 Install libraries and toolsets
 - 2.2.1 Checking and Upgrading the Host's Python Version
 - 2.2.2 Checking and Upgrading the Host's `make` Version
 - 2.2.3 Checking and Upgrading the Host's LZ4 Version
3. Setting Up Docker Environment
4. Software Development Guide
 - 4.1 Development Guide
 - 4.2 Chip Datasheet
 - 4.3 Debian Development Guide
 - 4.4 Third Party System Adaptation
 - 4.5 Software Update History
5. Hardware Development Guide
6. The Precaution of IO Power Design
7. SDK Configuration Framework Introduction
 - 7.1 SDK Project Directory Introduction
8. Introduction to Cross-Compilation Toolchain for SDK
 - 8.1 U-Boot and Kernel Compilation Toolchain
 - 8.2 Buildroot Toolchain
 - 8.2.1 Configuring the Compilation Environment
 - 8.2.2 Packaging Toolchain
 - 8.3 Debian Toolchain
 - 8.4 Yocto Toolchain
9. SDK Building Introduction
 - 9.1 SDK Compilation Commands
 - 9.2 SDK Compilation Command View
 - 9.3 SDK board-level configuration
 - 9.4 SDK Custom Configuration
 - 9.5 Fully Automated Compilation
 - 9.6 Module Compilation
 - 9.6.1 U-Boot Compilation
 - 9.6.2 Kernel Compilation
 - 9.6.3 Recovery Compilation
 - 9.6.4 Buildroot Compilation
 - 9.6.4.1 Building Buildroot Modules
 - 9.6.5 Debian Build
 - 9.6.6 Yocto Compilation
 - 9.7 Firmware Packaging
10. Upgrade Introduciton
 - 10.1 Windows Upgrade Introduction
 - 10.2 Linux Upgrade Instruction
 - 10.3 System Partition Introduction

1. SDK Precompiled Image

By using the RK3399 Linux SDK precompiled image, developers can bypass the process of compiling the entire operating system from source code. Instead, they can directly flash the precompiled image into the RK3399 development board, enabling rapid start of development and related evaluations and comparisons. This can reduce development time and cost wastage caused by compilation issues.

The firmware can be downloaded from the public address [Click here for SDK firmware download](#).

Firmware path: General Linux SDK Firmware -> Linux5.10 -> RK3399

If you need to modify SDK code or for a quick start, please refer to the following chapters.

2. Set up an Development Environment

2.1 Preparing the development environment

It is recommended to use Ubuntu 22.04 for compilation. Other Linux versions may need to adjust the software package accordingly. In addition to the system requirements, there are other hardware and software requirements. Hardware requirements: 64-bit system, hard disk space should be greater than 40G. If you do multiple builds, you will need more hard drive space

2.2 Install libraries and toolsets

When using the command line for device development, you can install the libraries and tools required for compiling the SDK by the following steps.

Use the following `apt-get` command to install the libraries and tools required for the following operations:

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2
```

Note:

The installation command is applicable to Ubuntu 22.04. For other versions, please use the corresponding installation commands based on the package names. If you encounter errors during compilation, you can install the corresponding software packages based on the error messages. Specifically:

- Python requires the installation of version 3.6 or higher, with version 3.6 used as an example here.
- Make requires the installation of version 4.0 or higher, with version 4.2 used as an example here.
- LZ4 requires the installation of version 1.7.3 or higher.
- Compiling Yocto requires a VPN network.

2.2.1 Checking and Upgrading the Host's Python Version

The method to check and upgrade the host's Python version is as follows:

- Check the host's Python version

```
$ python3 --version
Python 3.10.6
```

If the requirement of Python $\geq$ 3.6 is not met, you can upgrade by the following method:

- Upgrade to the new version `Python 3.6.15`

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 Checking and Upgrading the Host's `make` Version

The method to check and upgrade the host's `make` version is as follows:

- Checking the Host's `make` Version

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- Upgrading to a Newer Version of `make 4.2`

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 Checking and Upgrading the Host's LZ4 Version

The method to check and upgrade the host's LZ4 version is as follows:

- Check the host's LZ4 version

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

- Upgrade to a new version of LZ4

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

3. Setting Up Docker Environment

To assist developers in quickly completing the complex preparation work for the development environment mentioned above, we have also provided a cross-compiler Docker image to enable customers to quickly verify and thus reduce the time required to build the compilation environment.

Before using the Docker environment, you can refer to the following document for operations:

<SDK>/docs/en/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_EN.pdf.

The verified systems are as follows:

Distribution Version	Docker Version	Image Loading	Firmware Compilation
Ubuntu 22.04	24.0.5	pass	pass
Ubuntu 21.10	20.10.12	pass	pass
Ubuntu 21.04	20.10.7	pass	pass
Ubuntu 18.04	20.10.7	pass	pass
Fedora 35	20.10.12	pass	NR (not run)

The Docker image can be obtained from the website [Docker Image](#).

4. Software Development Guide

4.1 Development Guide

Aiming to help engineers get started with SDK development and debugging faster, We have released “Rockchip_Developer_Guide_Linux_Software_EN.pdf” with the SDK, please refer to the documents under the project's `docs/en/RK3399` directory.

4.2 Chip Datasheet

Aiming to help engineers get started with RK3399 development and debugging faster. We have released "Rockchip_RK3399_Datasheet_V2.1_20200323.pdf", please refer to the documents under the project's `docs/en/RK3399/Datasheet` directory.

4.3 Debian Development Guide

Aiming to help engineers get started with RK3399 Debian development and debugging faster, "Rockchip_Developer_Guide_Debian_EN.pdf" is released with the SDK, please refer to the documents under the project's `docs/en/Linux/System` directory, which will be continuously improved and updated.

4.4 Third Party System Adaptation

Aiming to help engineers get started with Third Party System Adaptation faster, "Rockchip_Developer_Guide_Third_Party_System_Adaptation_EN.pdf" is released with the SDK, please refer to the documents under the project's `docs/en/Linux/System` directory, which will be continuously improved and updated.

4.5 Software Update History

Software release version upgrade history can be checked through project xml file by the following command:

```
.repo/manifests$ realpath rk3399_linux5.10_release.xml
# e.g.:the printed version is v1.5.0 and the update time is 20240620
# <SDK>/.repo/manifests/rk3399_linux/rk3399_linux5.10_release_v1.5.0_20240620.xml
```

Software release version updated information can be checked through the project text file by the following command:

```
<SDK>/.repo/manifests/rk3399_linux/RK3399_Linux5.10_SDK_Note.md
or
<SDK>/docs/en/RK3399/RK3399_Linux5.10_SDK_Note.md
```

5. Hardware Development Guide

Hardware related development can refer to the user guide, which can be found in the engineering directory:

RK3399 Hardware Design Guidelines:

```
<SDK>/docs/en/RK3399/Hardware/Rockchip_RK3399_Hardware_Design_Guide_V1.2_EN.pdf
```

RK3399 IND Industry Board Hardware Development Guide:

6. The Precaution of IO Power Design

The IO level of the controller power domain must be consistent with the IO level of the connected peripheral chip, and the voltage configuration of software must be consistent with the voltage of hardware to avoid GPIO damage.



Note:

About matching of GPIO power domain and IO level:

PMUIO0_VDD, PMUIO1_VDD, VCCIO1_VDD, VCCIO2_VDD, VCCIO3_VDD, VCCIO4_VDD, VCCIO5_VDD, VCCIO6_VDD, VCCIO7_VDD, voltage of these GPIO power domain must be consistent with the IO level voltage of the connected peripheral to avoid GPIO damage.

Also need to note that the voltage configuration of software should be consistent with the voltage of hardware: For example, if hardware IO level is connected to 1.8V, the voltage configuration of software should be configured to 1.8V accordingly; if hardware IO level should be connected to 3.3V, and the voltage configuration of software should also be configured to 3.3V to avoid GPIO damage.

Please refer to the following documents for details:

<SDK>/docs/en/RK3399/Rockchip_RK3399_Introduction_IO_Power_Domains_Configuration.pdf
<SDK>/docs/en/Common/IO-DOMAIN/Rockchip_Developer_Guide_Linux_IO_DOMAIN_EN.pdf

7. SDK Configuration Framework Introduction

7.1 SDK Project Directory Introduction

There are buildroot, debian, recovery, app, kernel, u-boot, device, docs, external and other directories in the project directory. Repositories are managed using manifests, and the repo tool is used to manage each directory or its subdirectory corresponding to a git.

- buildroot: root file system based on Buildroot.
- debian: root file system based on Debian.
- device/rockchip: store board-level configuration for each chip and some scripts and prepared files for building and packaging firmware.
- docs: stores development guides, platform support lists, tool usage, Linux development guides, and so on.
- external: stores some third-party libraries, including audio, video, network, recovery and so on.
- kernel: stores kernel development code.
- output: stores the firmware information, compilation information, XML, host environment, etc. generated each time.

- prebuilts: stores cross-building toolchain.
- rkbin: stores Rockchip Binary and tools.
- rockdev: stores building output firmware.
- tools: stores some commonly used tools under Linux and Windows system.
- u-boot: store U-Boot code developed based on v2017.09 version.
- yocto: stores the root file system developed based on Yocto 4.0.

8. Introduction to Cross-Compilation Toolchain for SDK

Considering that the Rockchip Linux SDK is currently compiled only in the Linux PC environment, we have also only provided the cross-compilation toolchain for Linux. The prebuilt toolchain in the prebuilt directory is for use with U-Boot and Kernel. The specific Rootfs needs to be compiled with the corresponding toolchain for each, or a third-party toolchain can be used.

8.1 U-Boot and Kernel Compilation Toolchain

The SDK prebuilts directory contains pre-configured cross-compilers for U-Boot and Kernel compilation, as follows:

Directory	Description
prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu	GCC ARM 10.3.1 64-bit toolchain
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi	GCC ARM 10.3.1 32-bit toolchain

You can download the toolchain from the following address:

[Click here](#)

8.2 Buildroot Toolchain

8.2.1 Configuring the Compilation Environment

To compile individual modules or third-party applications, a cross-compilation environment must be set up. For example, for the RK3399, the cross-compilation tools are located in the `buildroot/output/rockchip_rk3399/host/usr` directory. The `bin/` directory and the `aarch64-buildroot-linux-gnu/bin/` directory need to be set as environment variables. Execute the script to automatically configure the environment variables in the top-level directory:

```
source buildroot/envsetup.sh rockchip_rk3399
```

Enter the command to check:

```
cd buildroot/output/rockchip_rk3399/host/usr/bin
./aarch64-linux-gcc --version
```

At this point, the following information will be printed:

```
aarch64-linux-gcc.br_real (Buildroot -gc307c95550) 12.3.0
```

8.2.2 Packaging Toolchain

Buildroot supports the packaging of the built-in toolchain into a compressed archive for third-party applications to compile independently. For detailed information on how to package the toolchain, please refer to the Buildroot official documentation:

```
buildroot/docs/manual/using-buildroot-toolchain.txt
```

In the SDK, you can directly run the following command to generate the toolchain package:

```
./build.sh bmake:sdk
```

The generated toolchain package is located in the `buildroot/output/*/images/` directory, named `aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz`, for users who have the need. After extraction, the path to `gcc` will be:

```
./aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-buildroot-linux-gnu-gcc
```

8.3 Debian Toolchain

Use Docker on the host machine, gcc, or `dpkg-buildpackage` for related compilations.

8.4 Yocto Toolchain

Refer to the following:

- [Building your own recipes from first principles](#)
- [New Recipe](#)

9. SDK Building Introduction

The SDK can be accessed through `make` or `./build.sh`, add target parameters to configure and compile related functions.

9.3 SDK board-level configuration

Enter the project `<SDK>/device/rockchip/rk3399` directory:

Board level configuration	Note
rockchip_rk3399_evb_ind_lpddr4_defconfig	Suitable for RK3399 industry development board
rockchip_rk3399_firefly_defconfig	Suitable for RK3399 firefly development boards
rockchip_rk3399_sapphire_excavator_lp4_defconfig	Suitable for RK3399 sapphire excavator with LPDDR4 development board
rockchip_rk3399_sapphire_excavator_defconfig	Suitable for RK3399 sapphire excavator development board

The first way:

Add board configuration file behind `/build.sh` , for example:

Select the board configuration of **RK3399 industry development board**:

```
./build.sh device/rockchip/rk3399/rockchip_rk3399_evb_ind_lpddr4_defconfig
```

Select the board configuration of the **RK3399 firefly development board**:

```
./build.sh device/rockchip/rk3399/rockchip_rk3399_firefly_defconfig
```

Select the board-level configuration of the **RK3399 sapphire excavator with LPDDR4 development board**:

```
./build.sh  
device/rockchip/rk3399/rockchip_rk3399_sapphire_excavator_lp4_defconfig
```

Select the board-level configuration of the **RK3399 sapphire excavator development board**:

```
./build.sh device/rockchip/rk3399/rockchip_rk3399_sapphire_excavator_defconfig
```

The second way:

```
rk3399$ ./build.sh lunch  
processing option: lunch  
  
You're building on Linux  
Lunch menu...pick a combo:  
  
0. default BoardConfig.mk  
1. BoardConfig-rk3399-evb-ind-lpddr4.mk  
2. BoardConfig-rk3399-firefly.mk  
3. BoardConfig-rk3399-sapphire-excavator-lp4.mk  
4. BoardConfig-rk3399-sapphire-excavator.mk  
5. BoardConfig.mk  
Which would you like? [0]:  
...
```

9.4 SDK Custom Configuration

The SDK can be configured through the `make menuconfig` command, with the main configurable components currently being as follows:

```
(rockchip_rk3399_evb_ind_lpddr4_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto) --->
[*] Loader (U-Boot) --->
[ ] AMPAK (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image) --->
    Boot (Android-style boot image) --->
[*] Recovery (based on Buildroot) --->
    Security feature depends on buildroot rootfs ***
    Extra partitions (oem, userdata, etc.) --->
    Firmware (partition table, misc image, etc.) --->
[*] Update (Rockchip update image) --->
    Others configurations --->
```

- **Rootfs:** The Rootfs here represents the "root file system," where you can choose different root file system configurations such as Buildroot, Yocto, Debian, etc.
- **Loader (U-Boot):** This is the configuration for the bootloader, typically U-Boot, which is used to initialize hardware and load the main operating system.
- **AMPAK:** A multi-core heterogeneous boot solution suitable for application scenarios requiring real-time performance.
- **Kernel:** Here you configure kernel options to customize the Linux kernel according to your hardware and application needs.
- **Boot:** Here you configure the support format for the Boot partition.
- **Recovery (based on buildroot):** This is the configuration for the recovery environment based on buildroot, used for system recovery and upgrades.
- **PCBA test (based on buildroot):** This is the configuration for a PCBA (Printed Circuit Board Assembly) test environment based on buildroot.
- **Security:** Enabling security features, including Secureboot methods, Optee storage methods, and writing Keys.
- **Extra partitions:** Used to configure additional partitions.
- **Firmware:** Here you configure firmware-related options.
- **Update (Rockchip update image):** Used to configure options for Rockchip's complete firmware.
- **Others configurations:** Other additional configuration options.

The `make menuconfig` configuration interface provides a text-based user interface to select and configure various options.

After the configuration is complete, use the `make savedefconfig` command to save these configurations so that the customized compilation will be based on these settings.

Through the above config, you can choose different Rootfs/Loader/Kernel configurations for various customized compilations, allowing you to flexibly select and configure system components to meet specific needs.

9.5 Fully Automated Compilation

To ensure a smooth update for each iteration of the Software Development Kit (SDK), it is recommended to clean up the previous compilation artifacts before updating. This practice helps to avoid potential compatibility issues or compilation errors, as old compilation artifacts may not be suitable for the new version of the SDK. To clean these artifacts, you can simply run the command `./build.sh cleanall`.

Navigate to the root directory of the project and execute the following commands to automatically complete all compilations:

```
./build.sh all # Only compiles the module code (u-Boot, kernel, Rootfs, Recovery)
               # Further execution of `./build.sh ./mkfirmware.sh` is required
               for firmware packaging

./build.sh     # Compiles the module code (u-Boot, kernel, Rootfs, Recovery)
               # Packages into a complete update.img upgrade package
               # All compilation information is copied and generated in the out
               directory
```

The default is Buildroot, but a different rootfs can be specified by setting the environment variable

`RK_ROOTFS_SYSTEM`. `RK_ROOTFS_SYSTEM` currently supports three systems: buildroot, debian, and yocto.

For example, to generate a debain system, the following commands can be used:

```
export RK_ROOTFS_SYSTEM=debian
./build.sh all-release
or
RK_ROOTFS_SYSTEM=debian ./build.sh all-release
```

Note:

Every time the SDK is updated, it is recommended to clean up the previous compiled products and run them directly `./build.sh cleanall`

9.6 Module Compilation

9.6.1 U-Boot Compilation

```
./build.sh uboot
```

9.6.2 Kernel Compilation

- Method One

```
./build.sh kernel
```

- Method Two

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-
x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
or
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
```

- Method Three

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
or
make ARCH=arm64 rk3399-evb-ind-lpddr4-linux.img -j
```

9.6.3 Recovery Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of Recovery.

```
<SDK># ./build.sh recovery
```

After compilation, a recovery.img will be generated in the Buildroot directory

```
output/rockchip_rk3399_recovery/images.
```

Note: The recovery.img includes the kernel, so every time there is a change in the Kernel, Recovery needs to be repackaged. The method to repackage Recovery is as follows:

```
<SDK># source buildroot/envsetup.sh
<SDK># cd buildroot
<SDK># make recovery-reconfigure
<SDK># cd -
<SDK># ./build.sh recovery
```

Note: Recovery is a non-essential feature, and some board-level configurations may not set it up.

9.6.4 Buildroot Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh rootfs
```

After compilation, different format images are generated in the `output/rockchip_rk3399/images` directory under the Buildroot directory, with the default format being rootfs.ext4.

For specific details, refer to the Buildroot development documentation:

```
<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Buildroot_EN.pdf
```

9.6.4.1 Building Buildroot Modules

Set up configurations for different chips and target features by using `source buildroot/envsetup.sh`.

```
$ source buildroot/envsetup.sh
Top of tree: rk3399

Pick a board:
24. rockchip_rk3399
25. rockchip_rk3399_base
26. rockchip_rk3399_recovery

Which would you like? [1]:
```

Default selection 24 `rockchip_rk3399`, then enter the Buildroot directory for RK3399 and start compiling the relevant modules.

For example, to compile the rockchip-test module, commonly used compilation commands are as follows:

Enter the buildroot directory

```
SDK#cd buildroot
```

- Delete and recompile rockchip-test

```
buildroot#make rockchip-test-recreate
```

- Recompile rockchip-test

```
buildroot#make rockchip-test-rebuild
```

- Delete rockchip-test

```
buildroot#make rockchip-test-dirclean
or
buildroot#rm -rf output/rockchip_rk3399/build/rockchip-test-master/
```

If you need to compile a single module or a third-party application, you need to configure the cross-compilation environment. For example, RK3399 its cross-compilation tool is located in the

`buildroot/output/rockchip_rk3399/host/usr` directory, you need to set the `bin/` directory of the tool and the `aarch64-buildroot-linux-gnu/bin/` directory as the environment variable, execute the script that automatically configures environment variables in the top-level directory::

```
source buildroot/envsetup.sh rockchip_rk3399
```

Enter the command to view:

```
cd buildroot/output/rockchip_rk3399/host/usr/bin
./aarch64-linux-gcc --version
```

The following information will be printed:

```
aarch64-linux-gcc.br_real (Buildroot) 12.3.0
```

Save to rootfs configuration file
buildroot\$ make update-defconfig

9.6.5 Debian Build

```
./build.sh debian
```

After compilation, generate linaro-rootfs.img in the Debian directory.

Description: re-install the depend packages
sudo apt-get install binfmt-support qemu-user-static live-build
sudo dpkg -i ubuntu-build-service/packages/*
sudo apt-get install -f

For specific details, please refer to Debian development documentation reference:

```
<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Debian_EN.pdf
```

9.6.6 Yocto Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh yocto
```

After compilation, the rootfs.img is generated in the yocto directory under build/lastest.

The default username for login is root. For more information on Yocto, please refer to the [Rockchip Wiki](#).

FAQ:

- If you encounter the following issue during the compilation above:

Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8). Python can't change the filesystem locale after loading so we need a UTF-8 when Python starts or things won't work.

Solution:

```
locale-gen en_US.UTF-8  
export LANG=en_US.UTF-8 LANGUAGE=en_US.en LC_ALL=en_US.UTF-8
```

Or refer to [setup-locale-python3](#)

- If you encounter git permission issues that cause compilation errors.

Due to the addition of the CVE-2022-39253 security detection patch in the newer version of git, if an older version of Yocto is required, poky should include the following to fix it:

```
commit ac3eb2418aa91e85c39560913c1ddfd2134555ba
Author: Robert Yang <liezhi.yang@windriver.com>
Date:   Fri Mar 24 00:09:02 2023 -0700
```

bitbake: fetch/git: Fix local clone url to make it work with repo

The "git clone /path/to/git/objects_symlink" couldn't work after the following change:

<https://github.com/git/git/commit/6f054f9fb3a501c35b55c65e547a244f14c38d56>

Alternatively, you can also revert the git version on the PC to V2.38 or an earlier version, such as the following:

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

9.7 Firmware Packaging

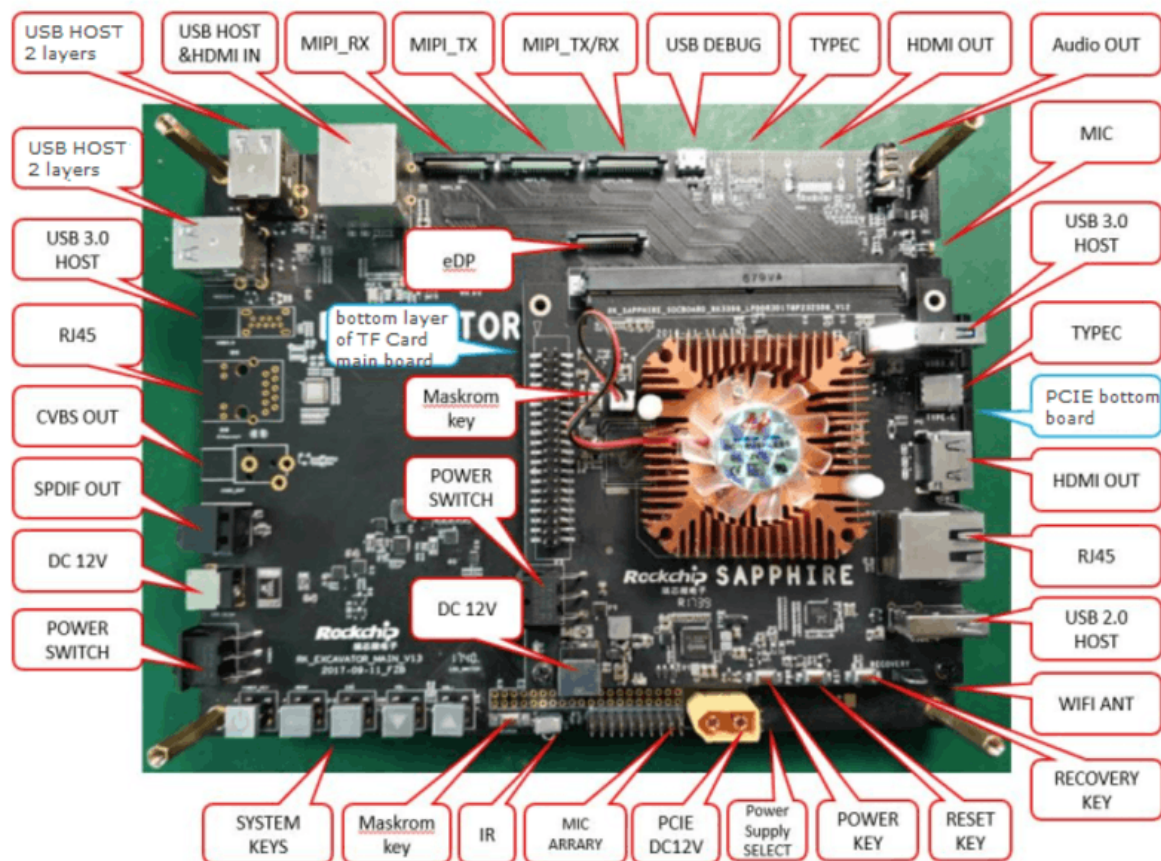
After compiling the various parts such as Kernel, U-Boot, Recovery, and Rootfs, navigate to the root directory of the project and execute the following command to automatically complete the packaging of all firmware to the `output/firmware` directory:

Firmware Generation:

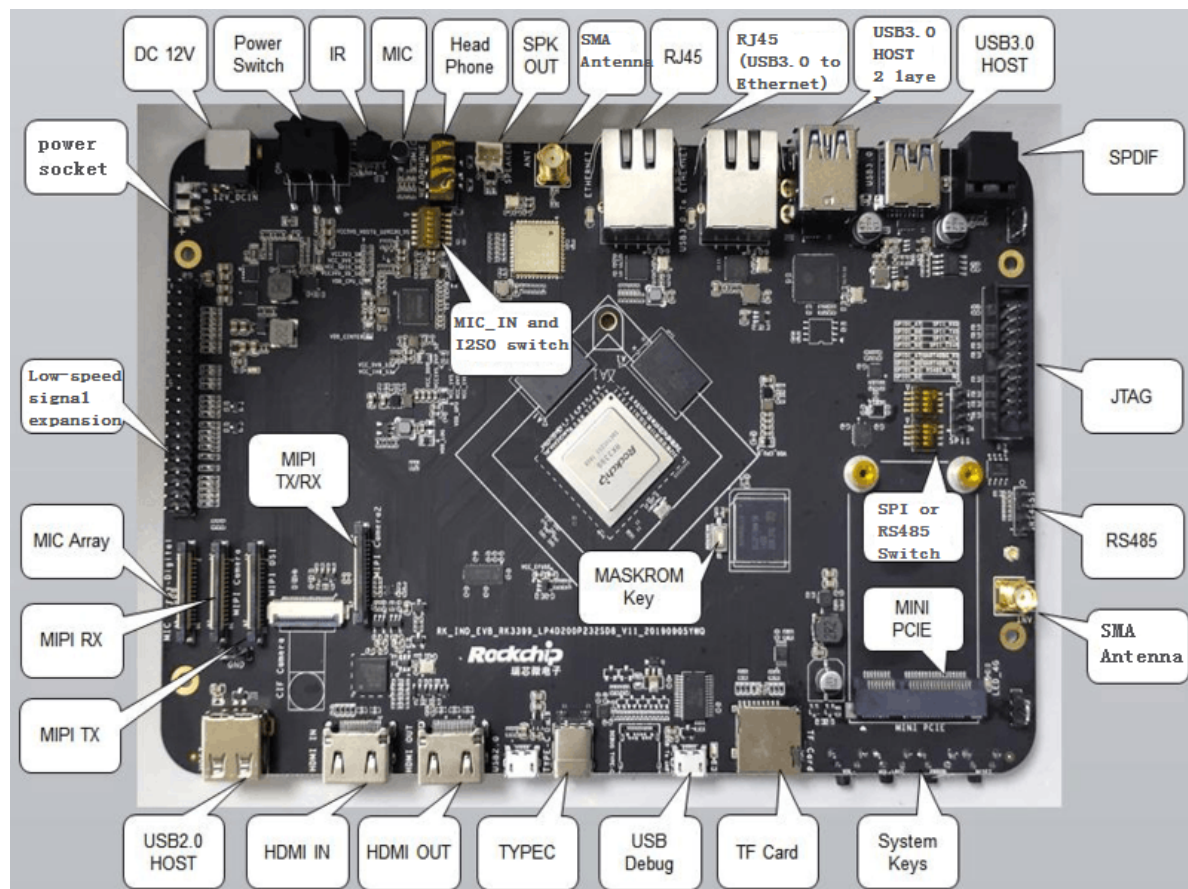
```
./build.sh firmware
```

10. Upgrade Introduciton

Interfaces layout of RK3399 excavator are showed as follows:



Interfaces layout of RK3399 IND industry board are showed as follows:

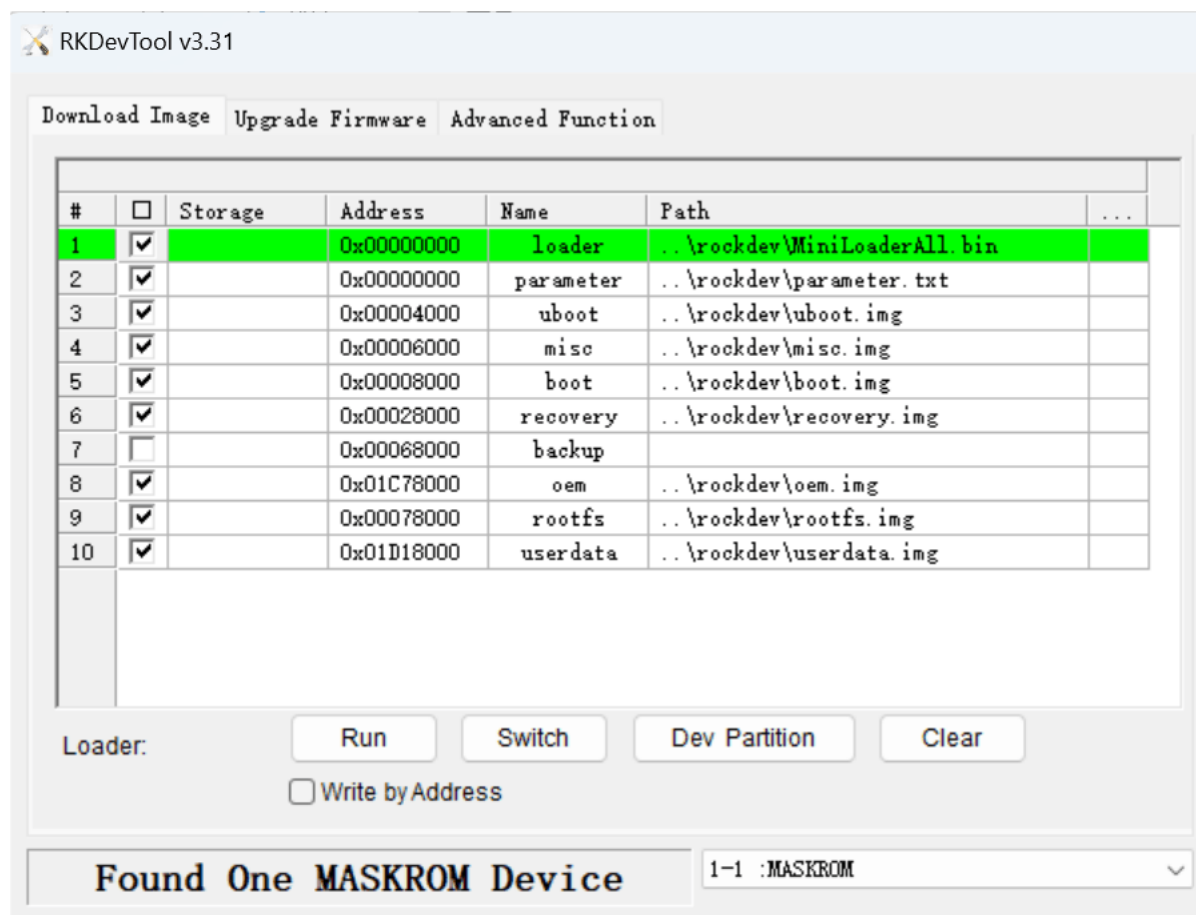


10.1 Windows Upgrade Introduction

SDK provides windows upgrade tool (this tool should be V3.15or later version) which is located in project root directory:

```
tools/
├── windows/RKDevTool
```

As shown below, after compiling the corresponding firmware, device should enter MASKROM or BootROM mode for update. After connecting USB cable, long press the button “MASKROM” and press reset button “RST” at the same time and then release, device will enter MASKROM Mode. Then you should load the paths of the corresponding images and click “Run” to start upgrade. You can also press the “recovery” button and press reset button “RST” then release to enter loader mode to upgrade. Partition offset and flashing files of MASKROM Mode are shown as follows (Note: Window PC needs to run the tool as an administrator):



Note: Before upgrade, please install the latest USB driver, which is in the below directory:

```
<SDK>/tools/windows/DriverAssitant_v5.12.zip
```

10.2 Linux Upgrade Instruction

The Linux upgrade tool (Linux_Upgrade_Tool should be v2.17 or later versions) is located in “tools/linux” directory. Please make sure your board is connected to MASKROM/loader rockusb, if the compiled firmware is in rockdev directory, upgrade commands are as below:

```

sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -trust rockdev/trust.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd

```

Or upgrade the whole update.img in the firmware

```

sudo ./upgrade_tool uf rockdev/update.img

```

Or in root directory, run the following command on the machine to upgrade in MASKROM state:

```

./rkflash.sh

```

10.3 System Partition Introduction

Default partition introduction (below is RK3399 IND reference partition):

Number	Start (sector)	End (sector)	Size	Name
1	16384	24575	4096K	uboot
2	24576	32767	4096K	trust
3	32768	40959	4096K	misc
4	40960	106495	32M	boot
5	106496	303104	32M	recovery
6	172032	237567	32M	bakcup
7	237568	368639	64M	oem
8	368640	12951551	6144M	rootfs
9	12951552	30535646	8585M	userdata

- uboot partition: for uboot.img built from uboot.
- trust partition: for trust.img built from uboot.
- misc partition: for misc.img built from recovery.
- boot partition: for boot.img built from kernel.
- recovery partition: for recovery.img built from recovery.
- backup partition: reserved, temporarily useless. Will be used for backup of recovery as in Android in future.
- oem partition: used by manufactor to store their APP or data, mounted in /oem directory
- rootfs partition: store rootfs.img built from buildroot or debian.
- userdata partition: store files temporarily generated by APP or for users, mounted in /userdata directory.

